Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

CSAI 203 • Fall 2026

Introduction to Software Engineering
Campus Event & Student Club Hub

Software Requirements Specification (SRS)

Team Number: Not yet assigned
Team Members:
Ahmed Amr — 202510874
Aya Selim — 202202258
Hamza Sami — 202511963
Seif Soltan — 202511598

Representative Contact:
s-ahmed.abdalaziz11@zewailcity.edu.eg

Date: October 2026

Contents

1. Introduction 3

1.1 Purpose 3

1.2 Scope 3

1.3 Definitions, Acronyms, and Abbreviations 3

1.4 References 3

1.5 Overview 3

2. Overall Description 3

2.1 Product Perspective 3

2.2 Product Functions 3

2.3 User Classes and Characteristics 3

2.4 Software Stakeholders 4

2.5 Operating Environment 4

2.6 Design and Implementation Constraints 4

2.7 User Documentation 4

2.8 Assumptions and Dependencies 4

3. Specific Requirements 4

3.1 Functional Requirements 4

3.2 Use Case Model 6

3.3 Domain Model 14

3.4 Non-Functional Requirements 16

3.5 External Interface Requirements 17

4. Appendices 17

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functions, user roles, and constraints of the Campus Event & Student Club Hub. It provides a basis for the project's design, implementation, and testing.

1.2 Scope

The Campus Event & Student Club Hub is a web application for managing student clubs and campus activities at Zewail City. Students can find events, register, access QR tickets, and submit feedback. Club managers organize events, record attendance, and track expenses. Administrators manage clubs, approve events, and allocate budgets.

1.3 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification; FR: Functional Requirement; NFR: Non-Functional Requirement; UML: Unified Modeling Language; QR: Quick Response; MVC: Model–View–Controller; RBAC: Role-Based Access Control; HTTPS: HTTP Secure; SMTP: Simple Mail Transfer Protocol; CSV: Comma-Separated Values.

1.4 References

CSAI 203 Course Project Description (Fall 2026); Dr. Mohamed Sami Rakha, Lecture 5: Requirements Engineering (UML Use Case IEEE Description Template, slides 64–67).

1.5 Overview

Section 2 describes the proposed system and its users. Section 3 covers functional requirements, use cases, domain classes, quality requirements, and interfaces. Section 4 contains the data dictionary and glossary.

2. Overall Description

2.1 Product Perspective

The proposed product is a standalone web application for student activities and club administration at Zewail City. It brings students, club managers, and administrators together in one system.

2.2 Product Functions

The main functions are account access, event creation and approval, event search and registration, QR ticketing, attendance tracking, club subscriptions, notifications, budget and expense management, and reporting.

2.3 User Classes and Characteristics

Students use the platform to explore events, manage registrations, and view tickets. Club managers create activities, check attendance, and record expenses. University administrators create clubs, assign managers, approve events, and manage budgets.

2.4 Software Stakeholders

Students need reliable event details, registration, and tickets. Club managers need accurate records of events, attendance, and expenses. University administrators need oversight of club activity and spending. The development team needs clear, testable requirements.

2.5 Operating Environment

The web application will support current versions of Chrome, Edge, and Safari on desktop and mobile devices. Internet access is required for registration, ticket validation, and database-backed operations.

2.6 Design and Implementation Constraints

The project must follow the Model–View–Controller (MVC) architecture and the technologies specified by the course. Automated code-generation tools are not permitted.

2.7 User Documentation

An FAQ will explain account access, event registration and cancellation, and digital ticket retrieval.

2.8 Assumptions and Dependencies

Users are assumed to have university email accounts and internet access. The system depends on database access and an email service for account recovery and event notifications.

3. Specific Requirements

3.1 Functional Requirements

FR-1 User Registration

Students and club managers shall be able to register using their full name, university email, and a secure password. Admin accounts are created by the system rather than through public registration. Scenario: A new student enters a Zewail City email on the sign-up page. The system checks the email domain and creates the account if the details are valid; an invalid domain produces an error.

FR-2 Authentication & Login

Registered users shall be able to sign in securely and request a password reset by email. Scenario: A user enters an email and password. If the credentials are valid, the system opens the dashboard for that role. If the user forgets the password, the system sends a recovery link to the registered address.

FR-3 Profile Management

Users shall be able to view and update permitted profile information, including a profile picture and major. Scenario: A student changes their major in Profile Settings. After validation, the updated major appears on the student’s profile.

FR-4 Club Profile Creation

Administrators shall be able to create club profiles with a name, logo, description, and assigned manager. Scenario: An administrator creates a profile for a new robotics club, uploads its logo, and assigns an eligible student as club manager.

FR-5 Event Publishing

Club managers shall be able to submit events with a title, date, time, location, capacity, and description. Scenario: A manager creates a coding workshop with a capacity of 50 seats. The system saves the event with Pending Approval status for administrative review.

FR-6 Event Modification

Club managers shall be able to edit or cancel events they manage. Canceling an event shall notify registered students. Scenario: When a speaker cancels a workshop, the manager cancels the event. The system marks the event as canceled and notifies all registered attendees.

FR-7 Event Dashboard

The dashboard shall list approved upcoming events so students can browse campus activities. Scenario: A student opens the dashboard and sees events ordered by date. Pending and rejected events are not shown.

FR-8 Filter and Search

Users shall be able to search by keyword and filter events by date range and club category. Scenario: A student chooses the Technology category and the current week. The dashboard displays events that match both filters.

FR-9 Event Registration

Logged-in students shall be able to register for an approved event when seats are available. The system shall check capacity before confirming registration. Scenario: A student selects Register for a workshop with five seats remaining, so the booking succeeds. If the workshop is full, the system prevents registration and displays Sold Out.

FR-10 Registration Cancellation

Students shall be able to cancel an event registration before the event starts. The canceled seat shall become available again. Scenario: A student cancels an existing registration because of a timetable conflict. The system updates the attendee list and releases one seat.

FR-11 Digital Ticket Generation

The system shall issue a digital ticket with a unique QR code after successful registration. Scenario: A student registers for the Science Fair. The ticket is generated and made available in the student dashboard for entry at the venue.

FR-12 Attendance Tracking

Club managers shall be able to scan QR tickets to record event attendance. Scenario: At the entrance to a workshop, the manager scans a student’s QR code with a phone. The system verifies the ticket and marks the student as Attended.

FR-13 Budget Allocation

Administrators shall be able to allocate a budget to a club, and club managers shall be able to view its allocated and remaining funds. Scenario: An administrator assigns the Art Club a 5,000 EGP budget. Its manager can then see the allocation in the financial dashboard.

FR-14 Expense Tracking

Club managers shall be able to record expenses, which the system shall deduct from the club's remaining budget. Scenario: The Art Club records 500 EGP spent on event supplies. The displayed remaining budget changes from 5,000 EGP to 4,500 EGP.

FR-15 Admin Approval Workflow

Administrators shall review submitted events and either approve or reject them before publication. Scenario: An administrator checks a pending Music Night event and approves it. The event status changes to Approved and it appears on the student dashboard.

FR-16 Club Subscription

Students shall be able to subscribe to club profiles to follow updates and activities. Scenario: After attending a literature club event, a student selects Subscribe on the club profile. The subscription is saved and the student is added to the club's followers.

FR-17 Notifications

The system shall notify students about upcoming registered events and updates from subscribed clubs. Scenario: When a literature club publishes a new event, students following that club receive a notification about it.

FR-18 Event Feedback

Students who attended a completed event shall be able to submit a rating and a short written review. Scenario: After a workshop ends, an attendee gives it four out of five stars and leaves a comment about the speaker.

FR-19 Calendar View

The system shall provide a monthly calendar of approved events to help students plan their schedules. Scenario: A student switches from the event list to the calendar to see the dates of upcoming activities in November.

FR-20 Reporting

Club managers and administrators shall be able to download attendance and expense reports. Scenario: At the end of the semester, an administrator generates a CSV file listing attendance totals and expenses for the selected events.

3.2 Use Case Model

The use case diagrams show how students, club managers, and administrators interact with the platform. Each use case is documented once, even when several roles use it.

3.2.1 Use Case Diagrams

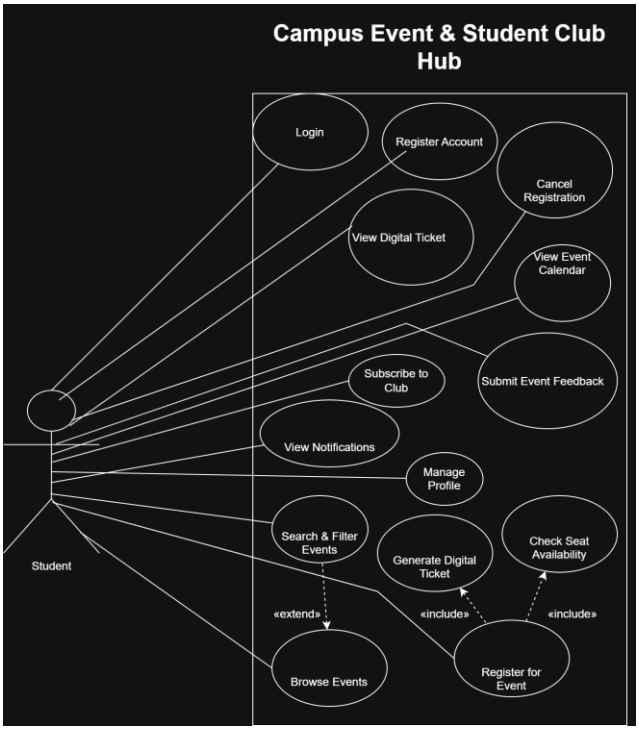


Figure 1: Student Use Case Diagram

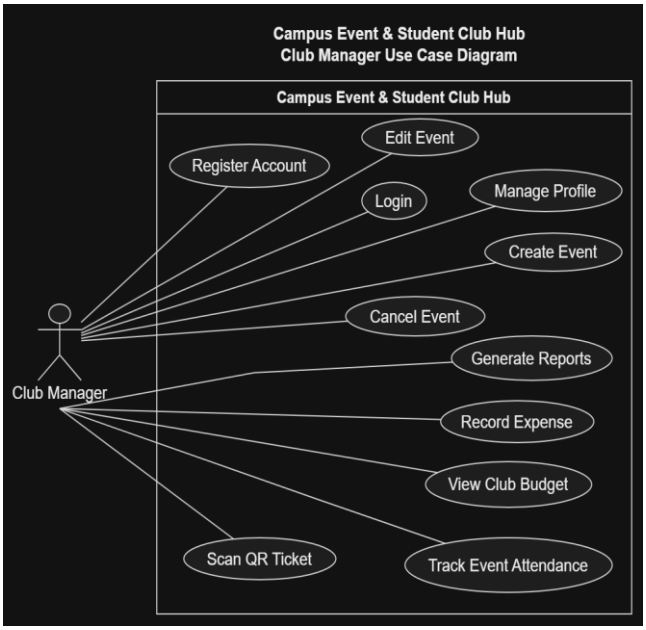


Figure 2: Club Manager Use Case Diagram

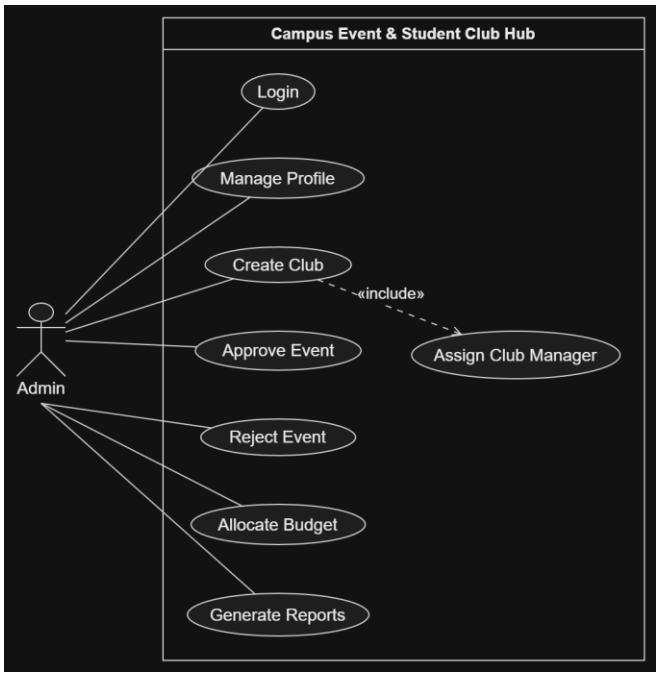


Figure 3: Admin Use Case Diagram

3.2.2 Use Case Descriptions

UC-01: Register Account

Primary actor: Student, Club Manager | Requirement: FR-1 | Trigger: opens Sign Up | Stakeholders: Applicants need access; admins need valid university identities | Preconditions: The user has a valid university email address and does not already have an account | Priority: High | Frequency: Per new user | Assumptions: University email validation is available

Main success scenario: (1) The user opens Sign Up; (2) enters a name, university email, and password; (3) the system validates the fields; (4) the system creates the account. | Alternative flow: If the email is invalid or already registered, the system displays an error message | Postconditions: A new account is created | Special requirements: University-domain email required | Include / extend: None

UC-02: Login

Primary actor: Student, Club Manager, Admin | Requirement: FR-2 | Trigger: selects Login | Stakeholders: Users need access; administrators require protected accounts | Preconditions: The user has an existing account | Priority: High | Frequency: Daily | Assumptions: Account authentication service is available

Main success scenario: (1) The user opens Login; (2) enters credentials; (3) the system verifies them; (4) the system opens the relevant dashboard. | Alternative flow: If the credentials are incorrect, the system displays an error message and allows another attempt | Postconditions: The user is logged into the system | Special requirements: Credentials verified securely | Include / extend: None

UC-03: Manage Profile

Primary actor: Student, Club Manager, Admin | Requirement: FR-3 | Trigger: opens Profile settings | Stakeholders: Users need accurate personal details | Preconditions: The user is logged into the system | Priority: Medium | Frequency: Occasional | Assumptions: The user has an authenticated session

Main success scenario: (1) The user opens Profile; (2) reviews the details; (3) updates permitted fields; (4) the system saves the changes. | Alternative flow: If the information is invalid, the system requests corrections | Postconditions: The user's profile information is updated | Special requirements: Only permitted profile fields may change | Include / extend: None

UC-04: Browse Events

Primary actor: Student | Requirement: FR-7 | Trigger: opens the event dashboard | Stakeholders: Students need to discover approved activities | Preconditions: The student can access the event dashboard | Priority: High | Frequency: Daily | Assumptions: Approved events are stored

Main success scenario: (1) The student opens the events page; (2) the system lists approved upcoming events; (3) the student selects an event to view its details. | Alternative flow: If no events are available, the system displays an empty list | Postconditions: The student can view available campus events | Special requirements: Only approved upcoming events shown | Include / extend: None

UC-05: Search and Filter Events

Primary actor: Student | Requirement: FR-8 | Trigger: submits search criteria | Stakeholders: Students need relevant events quickly | Preconditions: The event dashboard is accessible | Priority: Medium | Frequency: Frequent | Assumptions: Events are available to query

Main success scenario: (1) The student enters a keyword, date, or category; (2) the system applies the filters; (3) matching events are displayed. | Alternative flow: If no events match, the system displays a

message indicating that no results were found | Postconditions: The student can view events matching their preferences | Special requirements: Search by keyword, date, or category | Include / extend: None

UC-06: Register for Event

Primary actor: Student | Requirement: FR-9 | Trigger: selects Register on an event | Stakeholders: Students need confirmed seats; clubs need accurate counts | Preconditions: The student is logged in, and the event is approved and open for registration | Priority: High | Frequency: Per event | Assumptions: Event registration is open

Main success scenario: (1) The student selects an event and chooses Register; (2) the system checks capacity; (3) confirms registration; (4) generates a QR ticket. | Alternative flow: If the event is full, the system prevents registration | Postconditions: The student is registered and receives a digital ticket | Special requirements: Prevent overbooking | Include / extend: Includes UC-26 and UC-08

UC-07: Cancel Registration

Primary actor: Student | Requirement: FR-10 | Trigger: selects Cancel Registration | Stakeholders: Students need flexibility; clubs need accurate capacities | Preconditions: The student has registered for an event that has not started | Priority: High | Frequency: Occasional | Assumptions: The event has not started

Main success scenario: (1) The student opens registrations; (2) chooses Cancel; (3) confirms the action; (4) the system cancels registration and releases the seat. | Alternative flow: If the event has already started, the cancellation is rejected | Postconditions: The registration is canceled | Special requirements: Return canceled seat to capacity | Include / extend: None

UC-08: Generate Digital Ticket

Primary actor: Student (indirectly through event registration) | Requirement: FR-11 | Trigger: registration succeeds | Stakeholders: Students need unique entry tickets; clubs need verification | Preconditions: Event registration has been successfully completed | Priority: High | Frequency: Per registration | Assumptions: A confirmed registration exists

Main success scenario: (1) Registration succeeds; (2) the system creates a unique QR code; (3) saves the ticket; (4) makes it available on the student dashboard. | Alternative flow: If ticket generation fails, the system displays an error message | Postconditions: A digital ticket is available to the student | Special requirements: Ticket QR code must be unique | Include / extend: Included by UC-06

UC-09: View Digital Ticket

Primary actor: Student | Requirement: FR-11 | Trigger: selects View Digital Ticket | Stakeholders: Students need a ticket for event entry | Preconditions: The student has a confirmed event registration | Priority: High | Frequency: Per event | Assumptions: A ticket was generated

Main success scenario: (1) The student opens registered events; (2) selects a ticket; (3) the system displays its QR code. | Alternative flow: If no ticket is available, the system displays an appropriate

message | Postconditions: The student can access the ticket | Special requirements: Show the correct student ticket | Include / extend: None

UC-10: Subscribe to Club

Primary actor: Student | Requirement: FR-16 | Trigger: selects Subscribe on a club | Stakeholders: Students want club updates; clubs want followers | Preconditions: The student is logged into the platform | Priority: Medium | Frequency: Occasional | Assumptions: A club profile exists

Main success scenario: (1) The student opens a club profile; (2) selects Subscribe; (3) the system records the subscription and adds the student to the followers list. | Alternative flow: If the student is already subscribed, the system displays the existing subscription status | Postconditions: The student follows the selected club | Special requirements: Avoid duplicate subscriptions | Include / extend: None

UC-11: Submit Event Feedback

Primary actor: Student | Requirement: FR-18 | Trigger: selects Leave Feedback | Stakeholders: Attendees want to share ratings; clubs want comments | Preconditions: The student attended an event that has ended | Priority: Medium | Frequency: After attended events | Assumptions: Attendance can be checked

Main success scenario: (1) The attendee opens a completed event; (2) selects Feedback; (3) enters a rating and comment; (4) the system saves the review. | Alternative flow: If the event has not ended, feedback submission is not allowed | Postconditions: The feedback is recorded | Special requirements: Feedback only after attendance and event completion | Include / extend: None

UC-12: View Event Calendar

Primary actor: Student | Requirement: FR-19 | Trigger: opens Calendar View | Stakeholders: Students want to plan their schedules | Preconditions: The student can access the platform | Priority: Medium | Frequency: Weekly | Assumptions: Approved events have dates

Main success scenario: (1) The student opens Calendar; (2) the system displays approved events by date; (3) the student changes the month or opens event details. | Alternative flow: If no events are scheduled, the calendar remains empty for the selected period | Postconditions: The student can view events in calendar format | Special requirements: Calendar must match the approved event list | Include / extend: None

UC-13: Create Event

Primary actor: Club Manager | Requirement: FR-5 | Trigger: selects Create Event | Stakeholders: Managers want to submit activities; admins need review | Preconditions: The Club Manager is logged in and has permission to manage a club | Priority: High | Frequency: Per event | Assumptions: Manager has club permissions

Main success scenario: (1) The manager opens Create Event; (2) enters the required details; (3) submits the event; (4) the system validates it and saves it as Pending Approval. | Alternative flow: If required

information is missing, the system requests corrections | Postconditions: The event is saved with Pending Approval status | Special requirements: New events begin Pending Approval | Include / extend: None

UC-14: Edit Event

Primary actor: Club Manager | Requirement: FR-6 | Trigger: selects Edit Event | Stakeholders: Managers need to correct event information | Preconditions: The manager has permission to modify the selected event | Priority: Medium | Frequency: Occasional | Assumptions: The event belongs to the manager's club

Main success scenario: (1) The manager selects an event belonging to the club; (2) opens Edit; (3) changes the details; (4) the system validates and saves them. | Alternative flow: If invalid information is entered, the system requests corrections | Postconditions: The event information is updated | Special requirements: Validate changed event fields | Include / extend: None

UC-15: Cancel Event

Primary actor: Club Manager | Requirement: FR-6 | Trigger: confirms Cancel Event | Stakeholders: Managers need to cancel activities; attendees need notice | Preconditions: The selected event exists and has not already been canceled | Priority: High | Frequency: Occasional | Assumptions: The event is eligible for cancellation

Main success scenario: (1) The manager selects an event and chooses Cancel; (2) confirms the decision; (3) the system cancels the event and notifies attendees. | Alternative flow: If the manager cancels the request, the event remains unchanged | Postconditions: The event is canceled | Special requirements: Notify registered students after cancellation | Include / extend: None

UC-16: Track Event Attendance

Primary actor: Club Manager | Requirement: FR-12 | Trigger: opens Attendance dashboard | Stakeholders: Managers need to inspect event turnout | Preconditions: The manager is logged in, and the selected event has registered attendees | Priority: Medium | Frequency: On event days | Assumptions: Attendance records are accessible

Main success scenario: (1) The manager opens Attendance; (2) selects an event; (3) the system displays the current check-in statuses. | Alternative flow: If no attendees are registered, the system displays an empty list | Postconditions: The manager can monitor attendance | Special requirements: Respect club permissions | Include / extend: None

UC-17: Scan QR Ticket

Primary actor: Club Manager | Requirement: FR-12 | Trigger: scans a QR ticket | Stakeholders: Managers need valid check-ins; students need attendance records | Preconditions: The student has a valid digital ticket | Priority: High | Frequency: Each arrival | Assumptions: A ticket is presented at the entrance

Main success scenario: (1) The manager opens the scanner; (2) scans a ticket; (3) the system validates its QR code; (4) the system records attendance. | Alternative flow: If the ticket is invalid, the system rejects the check-in | Postconditions: The student's attendance is recorded | Special requirements: Reject invalid or reused tickets | Include / extend: None

UC-18: View Club Budget

Primary actor: Club Manager | Requirement: FR-13 | Trigger: opens Financial dashboard | Stakeholders: Managers need visibility into available funds | Preconditions: The manager is logged in and has access to the club's financial records | Priority: High | Frequency: Weekly | Assumptions: A manager has a designated club

Main success scenario: (1) The manager opens Finance; (2) the system retrieves the club budget and expenses; (3) the remaining balance is displayed. | Alternative flow: If no budget has been allocated, the system displays the available financial information | Postconditions: The manager can review the club's financial status | Special requirements: Show allocated, spent, and remaining funds | Include / extend: None

UC-19: Record Expense

Primary actor: Club Manager | Requirement: FR-14 | Trigger: submits a new expense | Stakeholders: Managers need to record club spending; admins need traceability | Preconditions: The manager has permission to record club expenses | Priority: High | Frequency: Per expense | Assumptions: The manager has club permissions

Main success scenario: (1) The manager enters the expense amount and description; (2) the system validates the entry; (3) records the expense; (4) updates the balance. | Alternative flow: If the entered amount is invalid, the system rejects the expense | Postconditions: The expense is recorded and the available budget is updated | Special requirements: Amount must be valid; update remaining balance | Include / extend: None

UC-20: Create Club

Primary actor: Admin | Requirement: FR-4 | Trigger: selects Create Club | Stakeholders: Admins need registered clubs with managers | Preconditions: The Admin is logged into the platform | Priority: High | Frequency: When founding a club | Assumptions: An eligible manager can be selected

Main success scenario: (1) The administrator opens Club Management; (2) enters club details; (3) assigns a manager; (4) the system creates the club. | Alternative flow: If required information is missing, the system requests corrections | Postconditions: A new club is created with an assigned manager | Special requirements: Club name and manager are required | Include / extend: Includes UC-21

UC-21: Assign Club Manager

Primary actor: Admin | Requirement: FR-4 | Trigger: selects Assign Manager | Stakeholders: Admins need clear management responsibility | Preconditions: The selected club exists, and an eligible manager is available | Priority: High | Frequency: Occasional | Assumptions: The candidate is eligible
Main success scenario: (1) The administrator selects a club and chooses Assign Manager; (2) selects an eligible user; (3) the system saves the assignment. | Alternative flow: If the selected user is not eligible, the assignment is rejected | Postconditions: The selected manager is assigned to the club | Special requirements: One assigned manager per club | Include / extend: Included by UC-20

UC-22: Approve Event

Primary actor: Admin | Requirement: FR-15 | Trigger: selects Approve on a pending event | Stakeholders: Admins need to control event publication | Preconditions: An event is waiting for administrative approval | Priority: High | Frequency: Per submitted event | Assumptions: The event is Pending Approval
Main success scenario: (1) The administrator opens pending events; (2) reviews an event; (3) approves it; (4) the system publishes the event. | Alternative flow: If the event is no longer pending, the system prevents duplicate approval | Postconditions: The event is approved | Special requirements: Only approved events appear publicly | Include / extend: None

UC-23: Reject Event

Primary actor: Admin | Requirement: FR-15 | Trigger: selects Reject on a pending event | Stakeholders: Admins need to decline unsuitable requests | Preconditions: An event is waiting for administrative approval | Priority: High | Frequency: Per submitted event | Assumptions: The event is Pending Approval
Main success scenario: (1) The administrator opens pending events; (2) selects an event; (3) rejects it; (4) the system marks it as Rejected and keeps it hidden. | Alternative flow: If the operation is canceled, the event remains pending | Postconditions: The event is rejected and is not visible to students | Special requirements: Rejected events must remain hidden | Include / extend: None

UC-24: Allocate Budget

Primary actor: Admin | Requirement: FR-13 | Trigger: submits a budget allocation | Stakeholders: Admins need to set club spending limits | Preconditions: The selected club exists | Priority: High | Frequency: Per allocation | Assumptions: A target club exists
Main success scenario: (1) The administrator opens Budget Management; (2) selects a club; (3) enters an allocation; (4) the system validates and saves it. | Alternative flow: If the amount is invalid, the system requests corrections | Postconditions: The club's budget is updated | Special requirements: Amount must be nonnegative | Include / extend: None

UC-25: Generate Reports

Primary actor: Club Manager, Admin | Requirement: FR-20 | Trigger: selects Generate Report | Stakeholders: Managers and admins need attendance and spending summaries | Preconditions: The user is logged in and has permission to access reports | Priority: Medium | Frequency: Monthly | Assumptions: Authorized records are available

Main success scenario: (1) The authorized user opens Reports; (2) selects attendance or expenses; (3) the system generates a CSV report; (4) the user downloads it. | Alternative flow: If no records are available, the system displays an appropriate message | Postconditions: The requested report is generated | Special requirements: CSV must reflect current records | Include / extend: None

UC-26: Check Seat Availability

Primary actor: Student (indirectly through event registration) | Requirement: FR-9 | Trigger: event registration checks capacity | Stakeholders: Students need seat confirmation; clubs must avoid overbooking | Preconditions: The student is logged into the system, and the selected event is approved and available for registration | Priority: High | Frequency: Each registration | Assumptions: The event has a capacity limit

Main success scenario: (1) Registration is requested; (2) the system retrieves capacity and confirmed registrations; (3) calculates available seats; (4) allows registration only when a seat is available. | Alternative flow: If the event is fully booked, the system prevents registration and displays a "Sold Out" message | Postconditions: Seat availability is verified before the student's registration is confirmed | Special requirements: No registration when full | Include / extend: Included by UC-06

UC-27: View Notifications

Primary actor: Student | Requirement: FR-17 | Trigger: opens Notifications | Stakeholders: Students need timely event and club updates | Preconditions: The student is logged into the system | Priority: Medium | Frequency: Daily | Assumptions: Notifications have been generated

Main success scenario: (1) The student opens Notifications; (2) the system lists reminders and updates; (3) the student selects an alert; (4) the system displays its details. | Alternative flow: If no notifications are available, the system displays a message indicating that there are no new notifications | Postconditions: The student can view notifications related to campus events and club activities | Special requirements: Only recipient notifications displayed | Include / extend: None

3.3 Domain Model

The conceptual model contains 13 classes for user roles, clubs, events, registrations, tickets, financial records, feedback, subscriptions, and notifications. Associations describe how these records relate to one another.

3.3.1 Conceptual Class Diagram

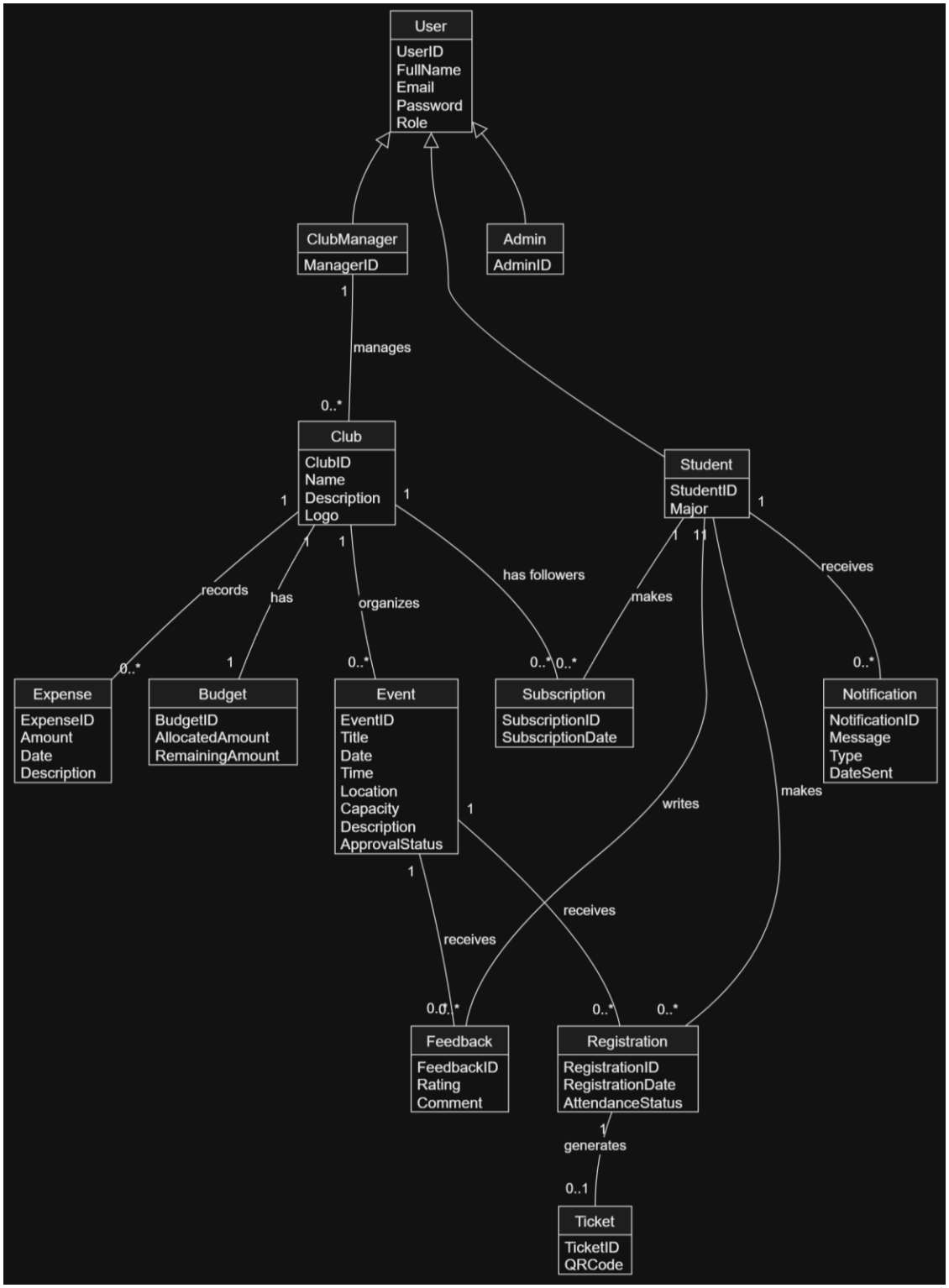


Figure 4: Conceptual Class Diagram

3.3.2 Class Descriptions

1. User — Represents a registered user of the platform. Attributes: UserID, FullName, Email, Password, Role. Student, ClubManager, and Admin inherit from User.

2. Student — Represents a university student who can register for events, receive tickets, and submit feedback. Attributes: StudentID, Major. A student can make multiple registrations and submit multiple feedback entries. Each registration and feedback entry belongs to one student.

3. ClubManager — Represents a user responsible for managing club activities. Attributes: ManagerID. A Club Manager can manage multiple clubs, while each club has one assigned manager.

4. Admin — Represents university staff responsible for supervising clubs and approving events. Attributes: AdminID. Admin inherits from User.

5. Club — Represents a university student club. Attributes: ClubID, Name, Description, Logo. Each club has one assigned manager and one budget. A club can organize multiple events and record multiple expenses.

6. Event — Represents an activity organized by a student club. Attributes: EventID, Title, Date, Time, Location, Capacity, Description, ApprovalStatus. Each event belongs to one club. An event can receive multiple registrations and feedback entries.

7. Registration — Represents a student's registration for an event. Attributes: RegistrationID, RegistrationDate, AttendanceStatus. Each registration belongs to one student and one event. A registration can have zero or one digital ticket, with a ticket issued after successful registration.

8. Ticket — Represents a digital ticket generated for an event registration. Attributes: TicketID, QRCode. Each ticket belongs to one registration and is used to verify attendance.

9. Budget — Represents the financial allocation assigned to a club. Attributes: BudgetID, AllocatedAmount, RemainingAmount. Each budget belongs to one club. The remaining amount is updated when club expenses are recorded.

10. Expense — Represents a cost recorded by a club manager. Attributes: ExpenseID, Amount, Date, Description. Each Expense belongs to one Club; a Club can record multiple expenses. Budget is represented separately and its remaining amount reflects recorded costs.

11. Feedback — Represents a student's rating and review of an event. Attributes: FeedbackID, Rating, Comment. Each feedback entry belongs to one student and one event and is submitted after attendance.

12. Subscription — Records a student following a club. Attributes: SubscriptionID, SubscriptionDate. Each record links one Student to one Club; both can have multiple subscriptions.

13. Notification — Stores an alert sent to a student. Attributes: NotificationID, Message, Type, DateSent. Each Notification belongs to one Student, who may receive many notifications.

3.4 Non-Functional Requirements

NFR-1 Security

All user passwords shall be hashed using a secure algorithm (e.g., bcrypt). Access to sensitive data and admin functions shall be restricted based on user roles.

Test: Attempt to access an admin-only URL using a standard student account. **| Pass:** The system denies access and returns a 403 Forbidden error.

NFR-2 Performance

The event dashboard shall load within 2 seconds under the stated test conditions. Following a successful registration, a digital ticket shall be generated and displayed for the student.

Test: Use browser developer tools (Network tab) to measure the load time of the dashboard after login. | **Pass:** Complete page load finishes in 2.0 seconds or less.

NFR-3 Usability

A new student shall be able to register for an event in no more than 3 clicks starting from the dashboard. The interface shall be responsive and usable on both laptop and mobile screens.

Test: Count the number of interactions required for a first-time user to secure a ticket from the homepage. | **Pass:** Registration takes no more than 3 clicks from the dashboard.

NFR-4 Availability

The system shall be available at least 99% of the time so students can access their tickets when needed.

Test: Monitor uptime during a 7-day test window. | **Pass:** The recorded system uptime is ≥ 99.0%.

NFR-5 Scalability

The backend shall support up to 1,000 concurrent users without crashing or noticeable slowdown, especially when major campus events are announced.

Test: Simulate 1,000 simultaneous user login and registration requests using a load-testing tool like JMeter. | **Pass:** At least 95% of requests complete within 5 seconds, with fewer than 1% errors.

3.5 External Interface Requirements

3.5.1 User Interface

The interface provides screens for browsing events, registering, viewing tickets, managing clubs, and administering the platform. Navigation and controls must remain readable on desktop and mobile screens.

3.5.2 Hardware Interface

Club managers use smartphone cameras to scan QR tickets and verify student attendance at event entrances.

3.5.3 Software Interface

An email service handles password recovery links, cancellation messages, and event notifications. The system stores relevant information in its database.

3.5.4 Communication Interface

The browser and server will communicate over HTTPS to protect credentials and other information transmitted by users.

4. Appendices

4.1 Appendix A: Data Dictionary

Users Table: Stores UserID, Full Name, University Email, Hashed Password, and Role (Student, Manager, Admin).

Clubs: ClubID, Name, Description, Logo, ManagerID.

Events Table: Stores EventID, Title, Date, Time, Location, Capacity, Description, ClubID, and Approval Status.

Registrations: RegistrationID, StudentID, EventID, RegistrationDate, AttendanceStatus.

Tickets: TicketID, RegistrationID, QRCode.

Budgets: BudgetID, ClubID, AllocatedAmount, RemainingAmount.

Expenses: ExpenseID, ClubID, Amount, Date, Description.

Feedback: FeedbackID, StudentID, EventID, Rating, Comment.

Subscriptions: SubscriptionID, StudentID, ClubID, SubscriptionDate.

Notifications: NotificationID, StudentID, Message, Type, DateSent.

4.2 Appendix B: Glossary

Digital Ticket: A system-generated record of registration represented visually as a QR code.

Dashboard: The main user interface feed displaying upcoming approved campus activities.

Role-Based Access: Security mechanism restricting system features based on whether the user is a Student, Manager, or Admin.